

Lesson 1 Notes: Special Matrices and Diagonal Storage

1. Main idea of the lesson

A normal matrix stores every value in the matrix.

A special matrix has a clear pattern. Because of that pattern, many values are either always zero or can be predicted. So we do not need to store every value.

Instead, we store only the important values in a 1D array and recreate the full matrix when needed.

In Lesson 1, the special matrix we study is the diagonal matrix.

2. What is a matrix?

A matrix is a table of numbers arranged in rows and columns.

Example:

```
3  0  0
0  7  0
0  0  5
```

This is a 3×3 matrix because it has:

```
3 rows
3 columns
```

A value inside a matrix is usually written as:

```
M[i, j]
```

Where:

```
i = row number
j = column number
```

For example:

```
M[1,1] = value in row 1, column 1  
M[2,2] = value in row 2, column 2  
M[3,3] = value in row 3, column 3
```

3. Matrix indexing: 1-based and 0-based

In the curriculum, matrix positions are written using 1-based indexing.

That means row and column numbers start at 1.

Example:

```
M[1,1]  
M[2,2]  
M[3,3]
```

But C and C++ arrays use 0-based indexing.

That means array indexes start at 0.

Example:

```
A[0]  
A[1]  
A[2]
```

This is why many formulas subtract 1.

Example:

```
Matrix position M[1,1] maps to array index A[0]  
Matrix position M[2,2] maps to array index A[1]  
Matrix position M[3,3] maps to array index A[2]
```

So:

```
array index = matrix row number - 1
```

4. What is a special matrix?

A special matrix is a square matrix with a useful pattern.

A square matrix has the same number of rows and columns.

Examples:

```
3 × 3  
4 × 4  
5 × 5
```

Special matrices save memory because their pattern allows us to avoid storing unnecessary values.

A special matrix may contain:

- many zeros
- repeated values
- mirrored values
- values only in certain positions

The curriculum mentions these special matrix types:

```
Diagonal matrix  
Lower triangular matrix  
Upper triangular matrix  
Symmetric matrix  
Tridiagonal matrix  
Band matrix  
Toeplitz matrix  
Sparse matrix
```

Lesson 1 focuses on the diagonal matrix.

5. What is a diagonal matrix?

A diagonal matrix is a matrix where only the main diagonal positions are important.

The main diagonal runs from the top-left corner to the bottom-right corner.

Example:

```
3  0  0  0
0  7  0  0
0  0  4  0
0  0  0  9
```

The diagonal positions are:

```
M[1,1]
M[2,2]
M[3,3]
M[4,4]
```

The pattern is:

```
i == j
```

That means the row number and column number are the same.

So:

```
If i == j, the position is on the diagonal.
If i != j, the position is outside the diagonal.
```

For a diagonal matrix, all non-diagonal positions are treated as zero.

In mathematical form:

```
M[i, j] = 0 if i != j
```

6. Why do we avoid storing the whole matrix?

Consider this 5×5 diagonal matrix:

```
3  0  0  0  0
0  7  0  0  0
0  0  4  0  0
0  0  0  9  0
0  0  0  0  6
```

A normal 2D array would store all 25 values.

Because:

```
5 × 5 = 25 cells
```

But only 5 values are useful:

```
3, 7, 4, 9, 6
```

The zeros outside the diagonal do not need to be stored because we already know they must be zero.

So instead of storing 25 values, we store only 5 values.

Compact storage:

```
A = [3, 7, 4, 9, 6]
```

This 1D array represents the full diagonal matrix.

7. Storage model for a diagonal matrix

For this matrix:

```
3  0  0  0  0
0  7  0  0  0
0  0  4  0  0
0  0  0  9  0
0  0  0  0  6
```

We store only:

```
A[0] = 3
A[1] = 7
A[2] = 4
A[3] = 9
A[4] = 6
```

The mapping is:

```
M[1,1] → A[0]
M[2,2] → A[1]
M[3,3] → A[2]
M[4,4] → A[3]
M[5,5] → A[4]
```

The index formula is:

```
index = i - 1
```

This formula is used only when:

```
i == j
```

If `i != j`, the value is not stored and should be treated as zero.

8. Why the formula is `i - 1`

Matrix positions start at 1:

```
M[1,1]
M[2,2]
M[3,3]
```

Array indexes start at 0:

```
A[0]
A[1]
A[2]
```

So we subtract 1 from the matrix row number.

Example:

```
M[1,1] → A[1 - 1] → A[0]  
M[2,2] → A[2 - 1] → A[1]  
M[3,3] → A[3 - 1] → A[2]
```

That is why the diagonal matrix index formula is:

```
index = i - 1
```

9. The `set` operation

The `set` operation stores a value in the matrix.

General form:

```
set(A, i, j, x)
```

Meaning:

```
Store value x at matrix position M[i,j]
```

For a diagonal matrix, we only store the value if the position is on the diagonal.

So we only store when:

```
i == j
```

If the position is not diagonal, we ignore the value.

Code

```
void set(int A[], int i, int j, int x) {  
    if (i == j) {  
        A[i - 1] = x;  
    }  
}
```

```
}  
}
```

Explanation

```
If i == j:  
    The position is diagonal.  
    Store x in A[i - 1].  
  
If i != j:  
    The position is not diagonal.  
    Do nothing.
```

10. Dry run of `set`

Example:

```
set(A, 3, 3, 4);
```

Step 1: Identify the values.

```
i = 3  
j = 3  
x = 4
```

Step 2: Check if the position is diagonal.

```
i == j  
3 == 3  
true
```

Step 3: Calculate the array index.

```
index = i - 1  
index = 3 - 1  
index = 2
```

Step 4: Store the value.


```
A[2] = 4
```

So the value 4 is stored as the third diagonal value.

11. Setting a non-diagonal value

Example:

```
set(A, 2, 5, 99);
```

Step 1: Check the position.

```
i == j  
2 == 5  
false
```

Since the position is not diagonal, the function does nothing.

Why?

Because in a diagonal matrix, every non-diagonal position must behave like zero.

So this position:

```
M[2, 5]
```

should remain zero.

The value 99 is not stored.

12. Structural zero

A structural zero is a zero that exists because of the matrix structure.

In a diagonal matrix, all non-diagonal positions are structural zeros.

Example:

```
M[2,5] = 0
```

This is not zero because the user stored zero there.

It is zero because the diagonal matrix rule says all non-diagonal positions must be zero.

This is important because structural zeros do not need memory.

The program can return zero without looking inside the array.

13. The `get` operation

The `get` operation returns the value at a matrix position.

General form:

```
get(A, i, j)
```

Meaning:

```
Return the value at M[i,j]
```

For a diagonal matrix:

```
If i == j, return A[i - 1].  
If i != j, return 0.
```

Code

```
int get(int A[], int i, int j) {  
    if (i == j) {  
        return A[i - 1];  
    }  
    return 0;  
}
```

Explanation

```
If the position is diagonal:  
    Look inside the 1D array.  
  
If the position is not diagonal:  
    Return 0 immediately.
```

14. Dry run of `get` for a diagonal position

Example:

```
get(A, 3, 3);
```

Step 1: Check the position.

```
i == j  
3 == 3  
true
```

Step 2: Calculate the array index.

```
index = i - 1  
index = 3 - 1  
index = 2
```

Step 3: Return the stored value.

```
return A[2]
```

15. Dry run of `get` for a non-diagonal position

Example:

```
get(A, 2, 5);
```

Step 1: Check the position.

```
i == j  
2 == 5  
false
```

Step 2: Return zero.

```
return 0
```

The function does not check the array.

It already knows the answer must be zero because `(2,5)` is not on the diagonal.

16. Full example

Suppose we store this array:

```
A = [3, 7, 4, 9, 6]
```

This represents the matrix:

```
3  0  0  0  0  
0  7  0  0  0  
0  0  4  0  0  
0  0  0  9  0  
0  0  0  0  6
```

Example 1

```
get(A, 4, 4)
```

Check:

```
4 == 4  
true
```

Calculate:

```
index = 4 - 1  
index = 3
```

Return:

```
A[3] = 9
```

So:

```
get(A, 4, 4) = 9
```

Example 2

```
get(A, 4, 1)
```

Check:

```
4 == 1  
false
```

Return:

```
0
```

So:

```
get(A, 4, 1) = 0
```

17. Memory saving

A normal $N \times N$ matrix stores:

```
 $N^2$  values
```

A diagonal matrix stores only:

N values

For N = 5:

Normal matrix storage = $5 \times 5 = 25$ values
Diagonal matrix storage = 5 values

For N = 1000:

Normal matrix storage = $1000 \times 1000 = 1,000,000$ values
Diagonal matrix storage = 1000 values

This is a huge memory saving.

The bigger the matrix becomes, the more useful special matrix storage becomes.

18. Operation complexity

Storage complexity

A diagonal matrix stores only N diagonal values.

So storage is:

$O(N)$

`set()` complexity

The `set()` function performs:

one condition check
one index calculation if needed
one array assignment if needed

So `set()` takes constant time:

$O(1)$

`get()` complexity

The `get()` function performs:

```
one condition check
one array lookup if needed
```

So `get()` takes constant time:

```
O(1)
```

Display complexity

Displaying the matrix means printing every row and every column.

For an $N \times N$ matrix, that means printing:

```
 $N^2$  positions
```

So display takes:

```
 $O(N^2)$ 
```

Even though storage is only $O(N)$, display is still $O(N^2)$ because the full visible matrix has N^2 positions.

19. Complexity table

Operation	Time Complexity	Space Complexity	Reason
Storage	-	$O(N)$	Store only N diagonal values.
<code>set()</code>	$O(1)$	-	One condition and one index formula.
<code>get()</code>	$O(1)$	-	One condition and one possible array lookup.
Display	$O(N^2)$	-	Prints every visible matrix position.

20. Beginner mental model

Think of the full matrix as a visual grid.

But the computer only stores the useful diagonal values.

Real stored array:

```
A = [3, 7, 4, 9, 6]
```

Full matrix view:

```
3  0  0  0  0
0  7  0  0  0
0  0  4  0  0
0  0  0  9  0
0  0  0  0  6
```

When someone asks for a diagonal value, the program looks inside the array.

Example:

```
M[3,3]
```

This is diagonal, so the program returns:

```
A[2]
```

When someone asks for a non-diagonal value, the program returns zero immediately.

Example:

```
M[3,5]
```

This is not diagonal, so the program returns:

```
0
```

The zero is not stored. It is created logically when needed.

21. Important formulas from Lesson 1

Diagonal condition

```
i == j
```

This means the position is on the main diagonal.

Non-diagonal condition

```
i != j
```

This means the position is outside the main diagonal.

Array index formula

```
index = i - 1
```

This is used only when `i == j`.

Get rule

```
if i == j:
    return A[i - 1]
else:
    return 0
```

Set rule

```
if i == j:
    A[i - 1] = x
else:
    do nothing
```

22. Common beginner mistakes

Mistake 1: Using `A[i]` instead of `A[i - 1]`

Wrong:

```
A[i] = x;
```

Correct:

```
A[i - 1] = x;
```

Why?

Because matrix positions start at 1, but array indexes start at 0.

Mistake 2: Storing non-diagonal values

Wrong idea:

```
Store every value the user gives.
```

Correct idea:

```
Store only values where i == j.
```

Non-diagonal values are structural zeros.

Mistake 3: Thinking display storage and actual storage are the same

The displayed matrix may look like this:

```
3  0  0
0  7  0
0  0  5
```

But the actual stored array is only:

```
A = [3, 7, 5]
```

The zeros are not stored.

23. Practice example

Given:

```
A = [10, 20, 30, 40]
```

This represents a 4×4 diagonal matrix.

Full matrix:

```
10  0  0  0
0   20 0  0
0   0  30 0
0   0  0  40
```

Questions:

```
get(A, 1, 1) = ?
get(A, 2, 2) = ?
get(A, 3, 4) = ?
get(A, 4, 1) = ?
```

Answers:

```
get(A, 1, 1) = 10
get(A, 2, 2) = 20
get(A, 3, 4) = 0
get(A, 4, 1) = 0
```

Reason:

Only positions where `i == j` are stored.

24. Exit question

Question:

```
Why does get(A, 4, 1) return 0 without using the array?
```

Answer:

Because `4 != 1`, so the position `(4, 1)` is not on the diagonal.

In a diagonal matrix, every non-diagonal position is a structural zero.

So the function already knows the answer is zero and does not need to read from the array.

25. Lesson 1 final summary

A diagonal matrix stores meaningful values only on the main diagonal.

The diagonal condition is:

```
i == j
```

All other positions are treated as zero:

```
i != j
```

Instead of storing all N^2 matrix values, we store only N diagonal values.

The storage array looks like this:

```
A[0], A[1], A[2], ...
```

The index formula is:

```
index = i - 1
```

The `set()` function stores a value only if the position is diagonal.

The `get()` function returns a stored value for diagonal positions and returns 0 for non-diagonal positions.

This is the foundation for the rest of the special matrix lessons. Later lessons use the same idea, but with different conditions and different index formulas.